

Java Programming Assignment

Sales Data Analysis — Product & ID-wise Sum

Submitted by: Student | Subject: Data Structures & Java

In this assignment, I have to work with sales data which has product names, IDs, prices and quantities. I need to calculate the total quantity sold for each product and each ID — first without using HashMap, then with HashMap. The data I used is taken from the table shown in class.

Sample Data Used

This is the data I am using in all four tasks. It has columns: ItemCode (ID), Product, Unit Price, Qty, Date, Rep.

```
// Sample Sales Data (I got this from the table on screen)
// ID      Product      UnitPrice  Qty    Date       Rep
// D       Product D    5          22     6/17/2019  E011
// D       Product D    5          24     6/23/2019  E011
// C       Product C    8          9      6/26/2019  E011
// C       Product C    8          6      6/29/2019  E011
// D       Product D    5          7      7/2/2019   E011
// D       Product D    5          15     7/4/2019   E011
// B       Product B    15         19     7/8/2019   E011
```

Task 1: Product-wise Sum — Bina HashMap (Without HashMap)

In this task, I am finding total quantity for each product name. I am NOT using HashMap here. I just used two arrays — one for product names and one for their total quantities. I loop through all sales and if product already exists in my array, I add qty to it. Otherwise I add new entry.

```
public class Task1_ProductWiseNoHashMap {

    public static void main(String[] args) {

        // my data arrays
        String[] products = {"Product D", "Product D", "Product C", "Product C",
                             "Product D", "Product D", "Product B"};
        int[] qty = {22, 24, 9, 6, 7, 15, 19};

        // I will store unique product names here
        String[] uniqueProducts = new String[10];
        int[] totalQty = new int[10];
        int count = 0; // how many unique products we found

        // loop through all sales records
        for (int i = 0; i < products.length; i++) {

            boolean found = false;

            // check if this product is already in our list
```

```

        for (int j = 0; j < count; j++) {
            if (uniqueProducts[j].equals(products[i])) {
                totalQty[j] = totalQty[j] + qty[i]; // add qty
                found = true;
                break;
            }
        }

        // if product not found, add it as new
        if (found == false) {
            uniqueProducts[count] = products[i];
            totalQty[count] = qty[i];
            count++;
        }
    }

    // print the result
    System.out.println("Product-wise Total Quantity (Without HashMap):");
    System.out.println("-----");
    for (int i = 0; i < count; i++) {
        System.out.println(uniqueProducts[i] + " --> " + totalQty[i]);
    }
}
}

```

Expected Output:

```

Product-wise Total Quantity (Without HashMap):
-----
Product D --> 68
Product C --> 15
Product B --> 19

```

Note: Product D total = 22+24+7+15 = 68. Product C = 9+6 = 15. Product B = 19. I checked manually also.

Task 2: ID-wise Sum — Bina HashMap (Without HashMap)

Same logic as Task 1 but this time I am grouping by ItemCode (ID) not product name. The IDs are D, C, B in our data. I loop through all records and accumulate qty per ID.

```

public class Task2_IDWiseNoHashMap {

    public static void main(String[] args) {

        // data
        String[] ids = {"D", "D", "C", "C", "D", "D", "B"};
        int[] qty = {22, 24, 9, 6, 7, 15, 19};

        // arrays to store unique ids and their totals
        String[] uniqueIDs = new String[10];
        int[] totalQty = new int[10];
        int count = 0;

        for (int i = 0; i < ids.length; i++) {

```

```

        boolean alreadyThere = false;

        // check if this id already exists
        for (int j = 0; j < count; j++) {
            if (uniqueIDs[j].equals(ids[i])) {
                totalQty[j] += qty[i]; // shortcut for totalQty[j] = totalQty[j] + qty[i]
                alreadyThere = true;
                break;
            }
        }

        if (!alreadyThere) {
            // new id found, store it
            uniqueIDs[count] = ids[i];
            totalQty[count] = qty[i];
            count++;
        }
    }

    // show results
    System.out.println("ID-wise Total Quantity (Without HashMap):");
    System.out.println("-----");
    for (int i = 0; i < count; i++) {
        System.out.println("ID: " + uniqueIDs[i] + " --> Total Qty: " + totalQty[i]);
    }
}
}

```

Expected Output:

```

ID-wise Total Quantity (Without HashMap):
-----
ID: D --> Total Qty: 68
ID: C --> Total Qty: 15
ID: B --> Total Qty: 19

```

Note: ID 'D' means Product D records. Same result as task 1 because IDs match products in this data.

Task 3: Product-wise Sum — HashMap ke Sath (Using HashMap)

Now we use HashMap! HashMap stores key-value pairs. Key = product name, Value = total qty. It is much easier than using two separate arrays. HashMap is in java.util package so we import it first. I learned that `getOrDefault()` method is very useful — it returns 0 if key not found yet.

```

import java.util.HashMap;

public class Task3_ProductWiseHashMap {

    public static void main(String[] args) {

        // same data as before
        String[] products = {"Product D", "Product D", "Product C", "Product C",
                             "Product D", "Product D", "Product B"};
        int[] qty = {22, 24, 9, 6, 7, 15, 19};
    }
}

```

```

// create a HashMap (product name -> total qty)
HashMap<String, Integer> myMap = new HashMap<String, Integer>();

// go through each record
for (int i = 0; i < products.length; i++) {

    String p = products[i];
    int q = qty[i];

    // getOrDefault returns 0 if product not in map yet
    int oldValue = myMap.getOrDefault(p, 0);

    // add current qty to old value and put back
    myMap.put(p, oldValue + q);
}

// print results
System.out.println("Product-wise Total Quantity (Using HashMap):");
System.out.println("-----");

// loop through all keys in the map
for (String key : myMap.keySet()) {
    System.out.println(key + " => " + myMap.get(key));
}
}
}

```

Expected Output:

```

Product-wise Total Quantity (Using HashMap):
-----
Product B => 19
Product C => 15
Product D => 68

```

Note: HashMap does not guarantee order of output. So the order might be different when you run it but the values will be same. HashMap made this task much easier than task 1!

Task 4: ID-wise Sum — HashMap ke Sath (Using HashMap)

This is same as Task 3 but with IDs (D, C, B) as the key instead of product names. HashMap makes it very easy — same logic, just different input array.

```

import java.util.HashMap;

public class Task4_IDWiseHashMap {

    public static void main(String[] args) {

        // our item IDs and qty
        String[] ids = {"D", "D", "C", "C", "D", "D", "B"};
        int[] qty = {22, 24, 9, 6, 7, 15, 19};

        // HashMap with ID as key and total qty as value

```

```

HashMap<String, Integer> idMap = new HashMap<String, Integer>();

for (int i = 0; i < ids.length; i++) {

    String currentID = ids[i];
    int currentQty = qty[i];

    // if this ID is not in map yet, getOrDefault gives 0
    int prevTotal = idMap.getOrDefault(currentID, 0);

    // now update the map with new total
    idMap.put(currentID, prevTotal + currentQty);
}

// display all id totals
System.out.println("ID-wise Total Quantity (Using HashMap):");
System.out.println("-----");
for (String id : idMap.keySet()) {
    System.out.println("ItemCode: " + id + " | Total Qty: " + idMap.get(id));
}
}
}

```

Expected Output:

```

ID-wise Total Quantity (Using HashMap):
-----
ItemCode: B | Total Qty: 19
ItemCode: C | Total Qty: 15
ItemCode: D | Total Qty: 68

```

Note: Again HashMap order might differ. But totals will always be correct.

Quick Comparison: With vs Without HashMap

Without HashMap	With HashMap
-----	-----
Need 2 arrays (names + totals)	Just 1 HashMap object
Manual searching with inner loop	getOrDefault() handles it
More lines of code	Less code, cleaner
$O(n^2)$ time complexity	$O(n)$ time (faster!)
Good for small data	Better for large data
No import needed	Need: <code>import java.util.HashMap</code>

What I Learned From This Task

1. Before HashMap, I did not know how to group data without it. Now I know we can use two arrays — one for keys and one for values. It works but it takes more code and is slower.
2. HashMap is really helpful. I just need to call `getOrDefault()` and `put()` and that is it! No need for inner loops to search.

3. The difference between product-wise and ID-wise grouping is just which column I use as the key. Same logic, different data.
4. I also learned that HashMap does not print in sorted order. If I want sorted output, I would need TreeMap (but that was not required here).
5. Writing code without HashMap first helped me understand what HashMap is actually doing inside. It is basically doing the same searching but much faster.
6. I should always import `java.util.HashMap` otherwise code gives error. I forgot this once and got a compile error, then I remembered.

Overall this assignment was good practice. At first I was confused about how to group without HashMap but then I figured out the two-array trick. HashMap version was definitely easier to write. I think in real projects, HashMap is always better to use.

— End of Assignment —